Mesh Generation & Optimization Algorithms Assignment

1. Domain

A random domain, NTUA campus was selected from Κτηματολόγιο

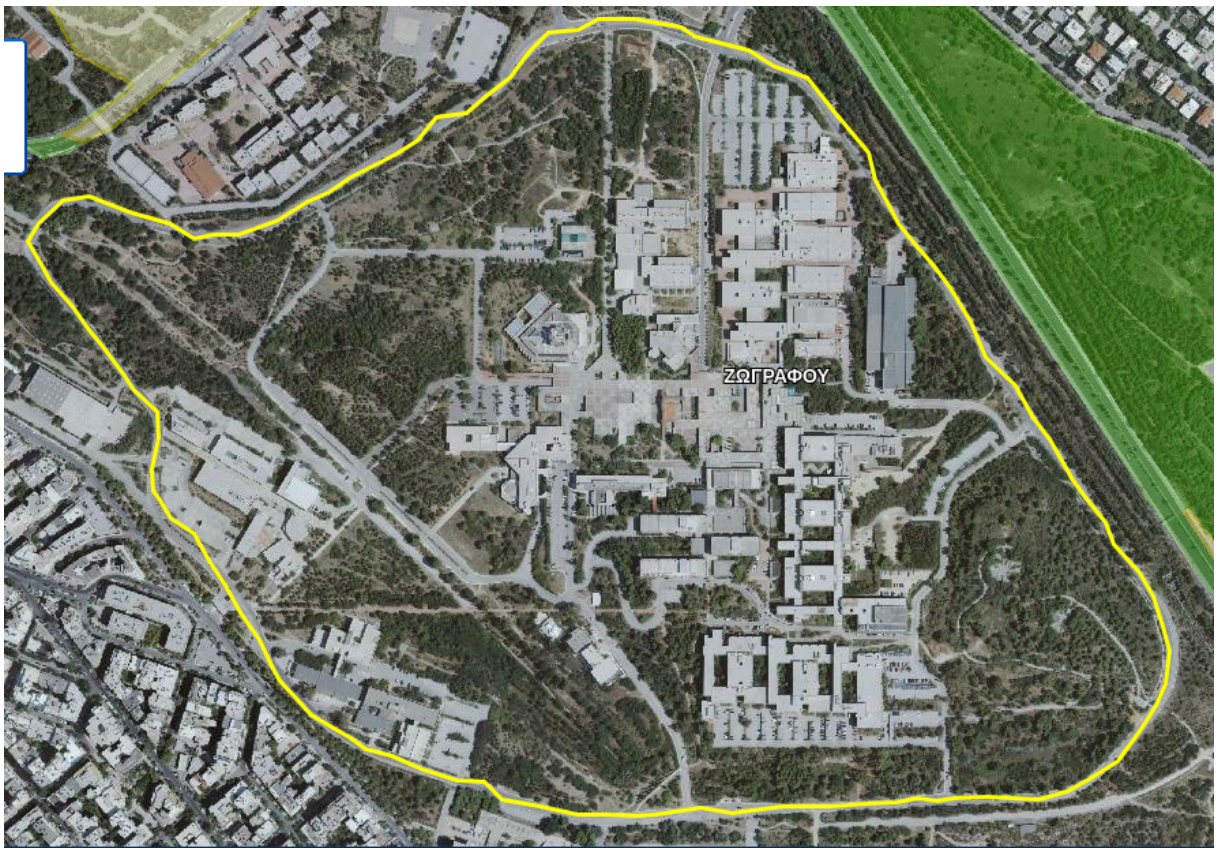


Figure 1: Photograph of the campus, with the designated boundaries

AA	latitude	longitude	AA	latitude	longitude	AA	latitude	longitude
0	481090.588	4202701.507	1	480750.598	4202680.34	2	480520.41	4202747.809
3	480459.556	4202791.466	4	480419.868	4202845.705	5	480411.931	4202881.424
6	480369.597	4202930.372	7	480344.462	4202950.216	8	480318.003	4202983.289
9	480323.295	4203071.924	10	480197.618	4203221.414	11	480193.649	4203242.581
12	480221.43	4203280.946	13	480303.451	4203265.071	14	480384.149	4203239.935
15	480463.524	4203273.008	16	480660.639	4203397.363	17	480680.483	4203438.373
18	480784.994	4203454.248	19	480946.39	4203402.654	20	481008.567	4203311.373
21	481307.547	4202870.841	22	481306.224	4202808.664	23	481248.016	4202737.226
24	481213.62	4202709.445	25	481213.62	4202709.445	26	481089.265	4202700.184
27	481089.265	4202700.184						

Plotting the points with Gnuplot, we get the domain of the problem.

2. Descritization

For the descitization part the algorithmic logic is as follows. We work in two different systems, $\{x, y\} \longleftrightarrow \{i, j\}$.

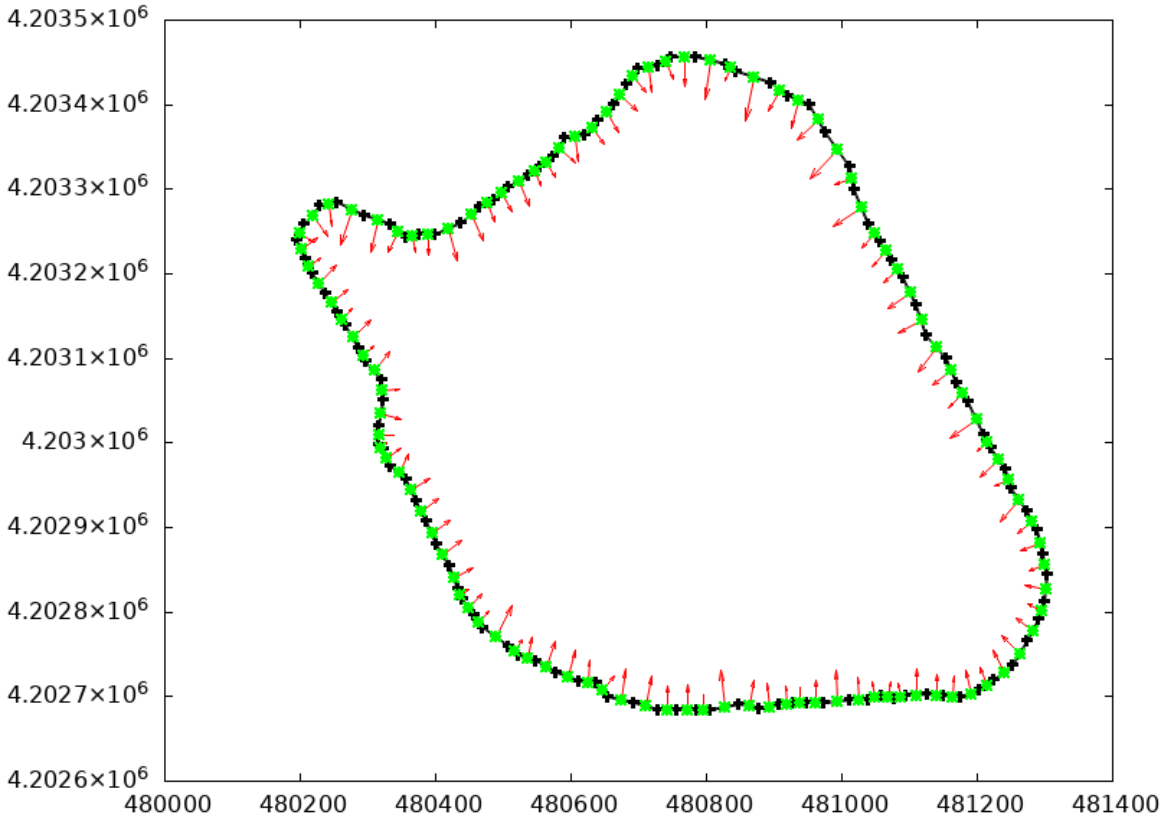


Figure 2: boundary domain, with normals and centers, for each egde

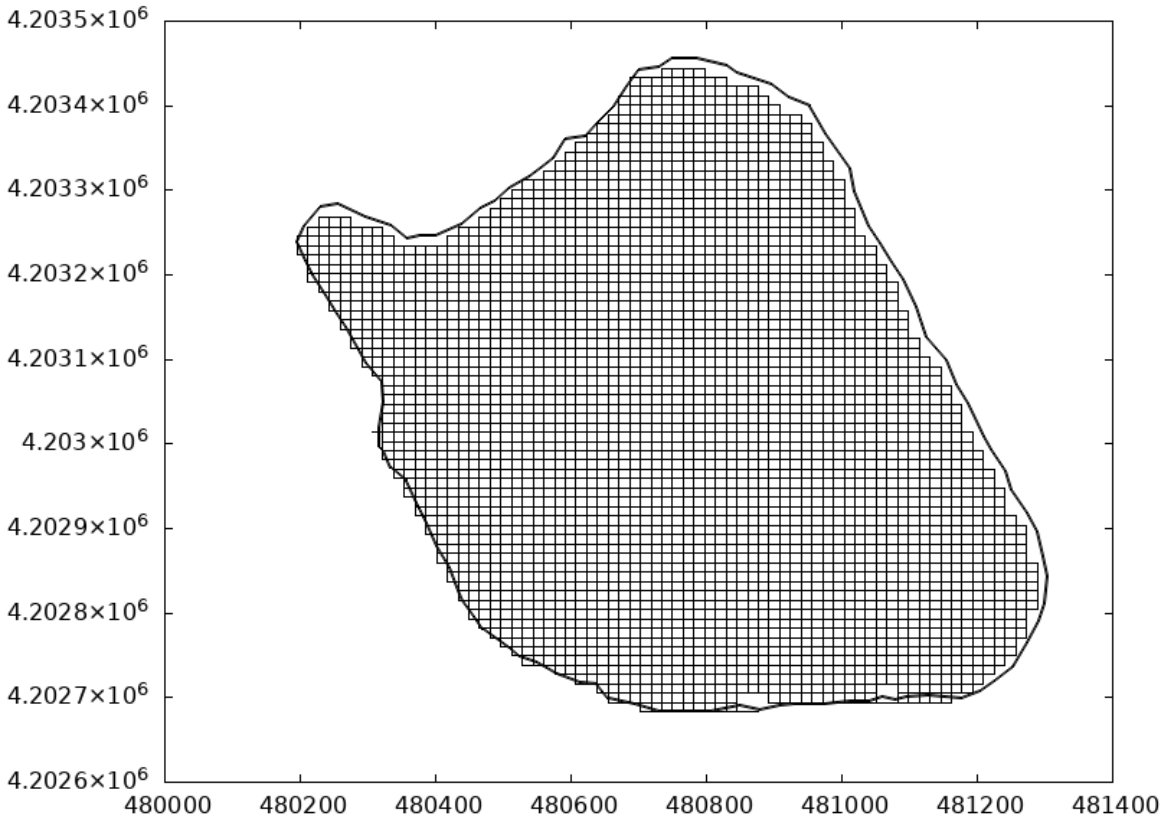
2.1. Subdivision

For each edge with compute the normalized coordinates $\{i, j\}$ and check wether the boundary on the grid is continuous. If it is not, the we recursively subdivide to ensure continuity. As an extra utility, a `.txt` an ASCII file is composed with the normalized coordinates of the boundary points of the domain.

When the continuous boundary in $\{i, j\}$ domain is generated, we iterate on the internal cells of that grid, and fill them as `true`.

```
for (int i = 0; i < grid.N; i++) {
  int lhs = 0;
  int rhs = grid.N-1;

  while (!grid.stencil_buffer[i*grid.N + lhs]) lhs++;  // advance
  while (!grid.stencil_buffer[i*grid.N + rhs]) rhs--;  // advance

  while (grid.stencil_buffer[i*grid.N + lhs]) lhs++;   // advance
  while (grid.stencil_buffer[i*grid.N + rhs]) rhs--;   // advance

  int a = Math.Min(lhs, rhs);
  int b = Math.Max(lhs, rhs);
  bool fill = true;

  switch (MeasureMarchingRow(grid.stencil_buffer, grid.N, i)) {
```

```
      case 2:
      case 3:
        for (int j = a; j <= b; j++) {
          grid.stencil_buffer[i*grid.N + j] = true;
        }
        break;

      case 4:
        for (int j = a; j <= b; j++) {
          if (grid.stencil_buffer[i*grid.N + j]) {
            while (grid.stencil_buffer[i*grid.N + j]) j++;  // advance
            j--;
            fill = !fill;
            continue;
          }

          if (fill) {
            grid.stencil_buffer[i*grid.N + j] = true;
          }
        }
      break;
      default: break;
    }
  }
```



For transforming from $\{x, y\} \rightarrow \{i, j\}$ the following snippet of code is used:

```
  /// <summary> transforms to i,j (Stencilsystem) from x,y (Cartesian system) </
summary>
  public static Index ToStencilSystem (int N, Vec2 p, BoundingBox bounds)
  {
    double x_min = bounds.v_min.X;
    double x_max = bounds.v_max.X;
    double y_min = bounds.v_min.Y;
    double y_max = bounds.v_max.Y;

    int i = (int)Math.Round((N-1)*(p.Y - y_min) / (y_max - y_min), 0); // normalize
to [0..N-1]
    int j = (int)Math.Round((N-1)*(p.X - x_min) / (x_max - x_min), 0); // normalize
to [0..N-1]

    return new Index{
      I = i,
      J = j,
    };
  }
```

While for the inverse transform $\{i, j\} \rightarrow \{x, y\}$:

```
  /// <summary> transforms to x,y (Cartensian system) from i,j (Stencil system) </
summary>
  public static Vec2 ToCartesianSystem (Grid grid, int i, int j)
  {
    double x_min = grid.bounds.v_min.X;
    double y_min = grid.bounds.v_min.Y;
    double dx = (grid.bounds.v_max.X - grid.bounds.v_min.X) / (double)grid.N;
```

```
    double dy = (grid.bounds.v_max.Y - grid.bounds.v_min.Y) / (double)grid.N;

    return new Vec2{
      X = (double)j * dx + x_min,
      Y = (double)i * dy + y_min,
    };
  }
```

3. Optimization

For the optimization part of the assignment the following function was used for the measuring the intesity of the signal from the antennas.

$$f_{\text{signal}}(x, y) = A \cdot \exp\left(-\left(\frac{(x - x_0)^2}{2\sigma_x^2} + \frac{(y - y_0)^2}{2\sigma_y^2}\right)\right)$$

A : amplitude

x : x-coordinate (practically the x-coordinate of the node of the grid)

x_o : x-coordinate of the center (practically the x-coordinate of the antenna)

y : y-coordinate (practically the y-coordinate of the node of the grid)

y_o : y-coordinate of the center (practically the y-coordinate of the antenna)

σ_x : variance on x -- (if I'm not mistaken)

σ_y : variance on y -- (better check it out to make sure)

Stepest Descend was utilized for the optimization of the antennas positions.

1. the antennas are assigned random positions inside the domain.
2. the intesity of the signal is calculated for each node of the mesh, as $s_{ij} = \sum_{i=1}^{K} f_{\text{signal}}(x_j, y_i)$
3. the gradient on each node (of the mesh) is computed.
4. The closest antenna position is displaced w.r.t the signal-intesity on that node and the gradient of that node.
5. after a fixed number of iterations `n_iter_max = 60`, the optimization loop is done and the positions of the antennas during the optimization, are stored into a gif, alongside a serialized text file.

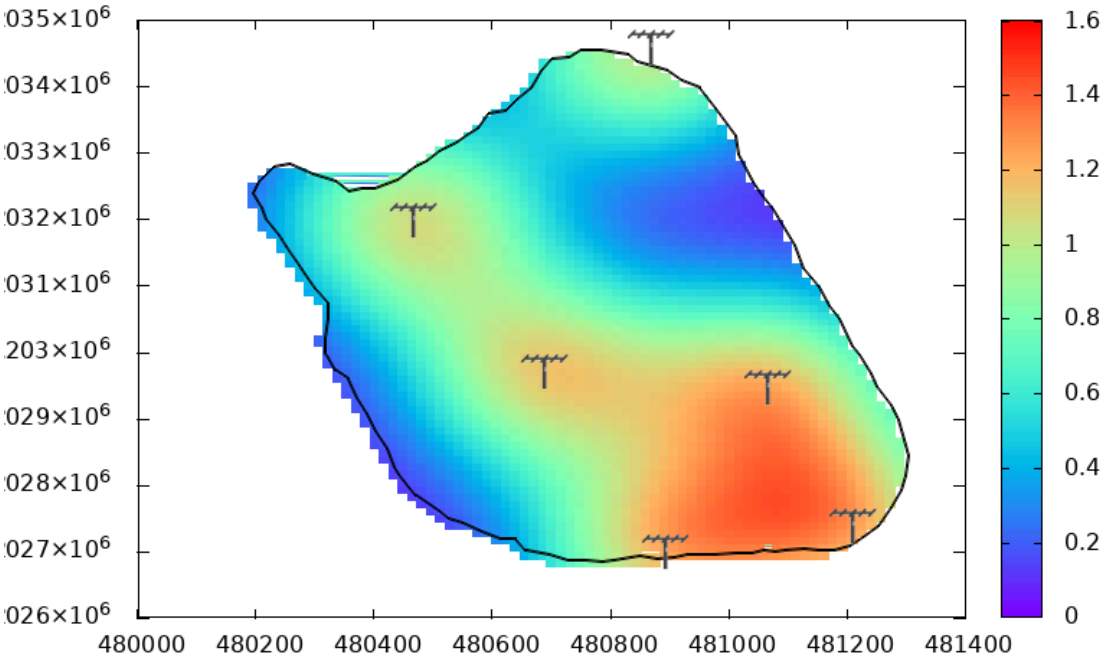


Figure 4: image of the final positions for the antennas and a color gradient for the intensity of the signal. The Optimization algorithm is NOT correct, it should be fixed. This image is a temporary placeholder. TO BE replaced, after fixes...**!!!**

4. TODO: Further improvements

Use a totally different datastructured for storing the physical values of the system. Instead of a sparse array, use a Quadtree instead (for 2D problems, or an Octree for 3D). One benefit of that method, is that it is has low foot-print on memory. It does not need to cache any coordinate $\{x, y\}$ values, the only essential information for the mesh, is the `stencil_buffer:[]bool` and the `bounds: .{v_min:Vec2, v_max:Vec2}` and the size `N` of the square matrix, mesh/grid. Everything else $\{x, y\}$ or $\{i, j\}$ are computed at runtime on the fly, extracted from the aforementioned fields. To get that approach one step further, is to use a Quadtree for the physical properties of the system. That, way a more compressed data structure, will make the algorithm even more memory efficient, as it fits to the design of the grid, so it *should* be a *straight-forward* implementation. With a Quadtree, a single value could be used for many nodes, where the difference is minimal, and more values when the difference is dense.